

MoMo SMS Analyzer

REST API Implementation Report

Team KSC

Chigozirim Menankiti · Samuel Nizeyimana · Mamashenge Kendra

Assignment 3 — API Security & Implementation

African Leadership University · May 2026

What is API Security?

API security is the practice of protecting Application Programming Interfaces from unauthorised access, data leakage, and malicious misuse. As APIs become the backbone of modern mobile and web applications — including financial services like MTN MoMo — securing them is no longer optional; a breach exposes sensitive transaction data for thousands of users.

Why it matters for MoMo SMS data

The MoMo SMS dataset contains personally identifiable information (PII): names, phone numbers, transaction amounts, and account balances. Exposing this data without authentication would violate Rwanda's data-protection regulations and expose users to financial fraud. Every API endpoint in this project is therefore gated behind an authentication layer.

Core security principles applied in this project

- **Authentication:** Only clients that supply valid credentials may access any endpoint. We implemented HTTP Basic Authentication as the baseline mechanism.
- **Authorisation:** Credentials map to user roles (admin, analyst). Future versions will enforce role-level permissions per endpoint.
- **Input Validation:** Every POST and PUT body is validated: required fields are checked, numeric fields are range-checked, and enumerated fields are verified against an allowlist.
- **Error Handling:** The API never leaks internal stack traces. Errors return structured JSON with a human-readable message and an appropriate HTTP status code.
- **Least Privilege:** The in-memory data store is isolated; no file-system write is exposed through the API.

Options & Stronger Alternatives

How Basic Authentication Works

HTTP Basic Authentication encodes the username and password as `base64(username:password)` and sends this string in the `Authorization` request header. The server decodes and compares the value against its credential store. If they match, the request is allowed.

Example header sent by the client:

```
Authorization: Basic YWRtaW46a3NjX2FkbWluMTIz
```

Why Basic Auth is Weak

Weakness	Explanation
Credentials on every request	Username + password are sent with every HTTP request. A single intercepted packet exposes
Base64 is not encryption	Base64 is trivially reversible. Without HTTPS/TLS, the credentials are effectively transmitted in
No token expiry	Basic Auth has no concept of session expiry. A leaked credential remains valid indefinitely until
No revocation without password change	To invalidate access for a compromised client, the server must change the password — affecti
Credential stuffing risk	A single static password is vulnerable to brute-force and credential-stuffing attacks at scale.

Recommended Stronger Alternatives

JWT (JSON Web Tokens)	The server issues a signed token on login. The client presents the token on subsequent requests — no password is re-transmitted. Tokens include an expiry (exp claim) and can be invalidated via a blacklist. Widely adopted for REST APIs.
OAuth 2.0 + OpenID Connect	Industry-standard delegated authorisation. Clients obtain short-lived access tokens from an authorisation server. Supports scopes (read-only vs admin), refresh tokens, and third-party identity providers (Google, GitHub). Ideal for multi-client, multi-service environments like MoMo.
API Key + HTTPS + Rate Limiting	Opaque random keys (not passwords) issued per client. Combined with TLS and per-key rate limiting, this is significantly stronger than Basic Auth and is common for machine-to-machine integrations.
mTLS (Mutual TLS)	Both client and server present certificates. No credential string is ever transmitted. The strongest option for high-security financial APIs, though operationally complex.

Base URL: <http://localhost:8080> · Authentication: [Basic Auth](#) · Content-Type: [application/json](#)

GET

/transactions

HTTP 200

List all 25 SMS transactions parsed from XML.

```
curl -u admin:ksc_admin123 http://localhost:8080/transactions
{"status":"success","count":25,"transactions":[...]}
```

GET

/transactions/{id}

HTTP 200

Return a single transaction by integer ID.

```
curl -u admin:ksc_admin123 http://localhost:8080/transactions/5
{"status":"success","transaction":{"id":5,"transactionType":"Airtime",...}}
```

POST

/transactions

HTTP 201

Create a new transaction. Required fields: transactionType, amount, rawMessage.

```
curl -u admin:ksc_admin123 -X POST -H "Content-Type: application/json" \
-d '{"transactionType":"Transfer","amount":12000,"rawMessage":"..."}' \
http://localhost:8080/transactions
{"status":"created","transaction":{"id":26,"transactionType":"Transfer",...}}
```

PUT

/transactions/{id}

HTTP 200

Partially update an existing transaction. Immutable fields (id, createdAt) are ignored.

```
curl -u admin:ksc_admin123 -X PUT -H "Content-Type: application/json" \
-d '{"transactionStatus":"Pending","fee":120}' \ http://localhost:8080/transactions/26
{"status":"updated","transaction":{"id":26,"transactionStatus":"Pending",...}}
```

DELETE

/transactions/{id}

HTTP 200

Permanently delete a transaction from the data store.

```
curl -u admin:ksc_admin123 -X DELETE http://localhost:8080/transactions/26
{"status":"deleted","message":"Transaction 26 successfully deleted."}
```

Error Code Reference

Code	Status	Triggered When
200	OK	Successful GET, PUT, DELETE
201	Created	Successful POST
400	Bad Request	Malformed request body or invalid ID format
401	Unauthorized	Missing or incorrect Basic Auth credentials
404	Not Found	Requested transaction ID does not exist

422	Unprocessable Entity	Required fields missing or values fail validation
-----	----------------------	---

Algorithms — Benchmark Results

Experimental Setup

We compared two search strategies for retrieving a transaction by its integer ID over all 25 records from the XML dataset. Each search was repeated 100,000 times per ID to obtain stable nanosecond averages, eliminating OS scheduling noise.

Results — Linear Search vs Dictionary Lookup

IDType	Linear (ns)	Dict (ns)Speedup
1 Incoming	200.9	81.5 2.5×
3 Withdrawal	196.0	82.2 2.4×
5 Airtime	243.6	81.0 3.0×
8 Bundle_Purchase	348.9	81.2 4.3×
10 Incoming	384.7	81.5 4.7×
12 Transfer	436.8	81.3 5.4×
15 Incoming	512.8	80.4 6.4×
18 Payment	593.3	81.5 7.3×
20 Bundle_Purchase	646.5	80.6 8.0×
23 Withdrawal	729.7	80.6 9.1×
25 Payment	779.3	81.3 9.6×
AVG(all 25)	463.3	81.5 5.7×

Note: linear search time grows proportionally with ID position ($O(n)$). Dictionary lookup remains constant at ~81 ns regardless of position ($O(1)$).

Why Dictionary Lookup is Faster

Linear search must compare each element's id field sequentially until a match is found. In the best case (ID=1) this takes one comparison; in the worst case (ID=25) it takes 25. Average time therefore grows as $O(n)$. As our benchmark confirms, linear search time for ID 25 (~779 ns) is roughly 4× that of ID 1 (~201 ns) — consistent with $O(n)$ scaling.

Python's dict is implemented as a hash table. When we call dict.get(id), Python computes hash(id) in constant time and directly indexes into the hash array. The result is retrieved in $O(1)$ time regardless of the dataset size. Our benchmark shows this averaging ~81 ns whether accessing ID 1 or ID 25 — the time does not grow with position.

Alternative Data Structures

Structure	Complexity	Best Use Case
Binary Search (sorted list)	$O(\log n)$	Efficient for ordered access (e.g., date-range queries). Needs data sorted by
B-Tree / B+ Tree	$O(\log n)$	Used internally by SQL databases. Excellent for mixed point and range queri
Trie (Prefix Tree)	$O(m)$, m =key length	Optimal for prefix-based search on string keys like externalTransactionId.
Hash Table (our choice)	$O(1)$ average	Best for exact integer ID lookup in memory. Constant time regardless o

What We Built

Team KSC built a fully functional REST API in plain Python using only the standard library's `http.server` module — no external frameworks. The API parses 25 MoMo SMS transactions from XML, exposes full CRUD operations, enforces Basic Authentication on every endpoint, and returns structured JSON with appropriate HTTP status codes.

Security Trade-offs

Basic Authentication was implemented as required by the assignment specification. In a real-world deployment this would be replaced with JWT or OAuth 2.0. Specifically, JWT tokens would eliminate the need to transmit credentials on every request, provide expiry semantics, and support token revocation — all critical properties for a financial data API handling sensitive MoMo transaction records.

DSA Insight

The benchmark confirmed the theoretical difference between $O(n)$ and $O(1)$ algorithms in practice. At just 25 records the dictionary is $5.7\times$ faster on average; at 1,000 records the gap would grow to roughly $200\times$ for worst-case lookups. This directly motivates why production databases always maintain index structures (B-Trees or hash indexes) rather than performing full table scans for primary-key lookups.

Future Improvements

- Replace Basic Auth with JWT bearer tokens (e.g. using PyJWT).
- Add persistent storage (SQLite or PostgreSQL) to survive server restarts.
- Implement pagination and filtering (GET `/transactions?type=Incoming&page;=2`).
- Add HTTPS/TLS termination (e.g. via nginx reverse proxy).
- Write automated unit tests with pytest and test the authentication middleware.
- Role-based access control: restrict DELETE to admin, restrict GET to analyst.